Article#1

From 3GB to 8MB: What MRL + Binary Quantization Actually Costs in Retrieval Quality

We've all been there. You start with a modest collection of embeddings for downstream classification or search. **10,000 vectors?** Perfectly manageable. Everything is snappy, the latency is negligible, and life is reasonably good.

Then the product scales—which is brilliant—until the invoice arrives. Suddenly, you're looking at RAM requirements of **3 GB to 600 GB** just to keep **100M vectors** "awake." That is before factoring in model weights, index overhead, or the application itself. It is a hefty price to pay for a bit of semantic search.

Let’s look at the arithmetic

Most standard embedding models output vectors with 768 or 1,536 dimensions. If we’re storing these as float32 (which is the typical default) We’re looking at 4 bytes per dimension. The arithmetic is fairly straightforward, if a bit sobering:

$$\text{768-dim float32} \rightarrow 4 \times 768 = 3{,}072 \text{ bytes per vector} \approx 3 \text{ KB}$$

$$\text{1,536-dim float32} \rightarrow 4 \times 1{,}536 = 6{,}144 \text{ bytes per vector} \approx 6 \text{ KB}$$

Scaled up:

$$\text{1M vectors} \times 3{,}072 \text{ B} = 3.07 \text{ GB} \quad \text{(768-dim)}$$

$$\text{100M vectors} \times 3{,}072 \text{ B} = 307 \text{ GB} \quad \text{(768-dim)}$$

$$\text{100M vectors} \times 6{,}144 \text{ B} = 614 \text{ GB} \quad \text{(1,536-dim)}$$

3 KB per vector seems harmless. 300 GB of RAM does not. And storage is not the only problem; it affects everything it touches, and it touches everything.

Why We Cannot Simply Put It on S3

The instinct is to store vectors cheaply on S3 or disk and load only what is needed. Sounds reasonable. Does not work.

Vector search requires the entire index to be in RAM simultaneously for fast random access. HNSW and IVF, the index structures used by FAISS and most vector databases, traverse a graph that jumps unpredictably across millions of vectors. You cannot stream a graph. You cannot chunk it. The whole thing must live in RAM, always, ready for random access.

This is what I call the **RAM Tax.** There is no choice of a cheaper memory tier. The index decides for you.

The index also adds overhead on top of raw storage. HNSW typically adds 50 to 100% on top. 300 GB of float32 vectors becomes 450 to 600 GB of indexed data in RAM, permanently. Think of it as a very expensive subscription you forgot to cancel.

Why Batching Does Not Help

The instinct when something is slow is to batch it. Batching helps for *generation*; encoding 1,000 documents at once is faster than one by one because the GPU works in parallel. Batching does not help with *search*.

The index is not a sequence. It is a graph. Graph traversal requires the whole graph in memory. You cannot search 100M vectors in batches of 10K the way you train on 100M examples in batches of 10K.

Why VRAM Does Not Help Either

Three problems:

First, GPU VRAM is small and already occupied. The embedding model lives there. An A100 has 80 GB, already claimed by model weights, KV cache, and inference buffers. There is no room for 300 GB of vectors.

Second, vector search is a CPU job. The GPU encodes the query. Everything after that, ANN index traversal, similarity computation across the corpus, runs on CPU, reading from RAM. The GPU is idle during search, earning its keep by doing absolutely nothing.

Third, even if vectors could be moved to VRAM, the PCIe bus is the ceiling. PCIe 4.0 delivers approximately 32 GB/s of bandwidth. Moving 300 GB from CPU RAM to GPU VRAM takes:

$$t = \frac{300 \text{ GB}}{32 \text{ GB/s}} \approx 9.4 \text{ seconds per query}$$

That is not feasible. Vectors live in CPU RAM. Not a choice; a constraint

The Problem Compounds at Every Layer

Layer 1 — Storage and RAM

To hold 600 GB with reasonable headroom, you need memory-optimised cloud instances. On AWS:

$$4.03/\text{hr} \times 2 \text{ instances} \times 24 \text{ hrs} \times 30 \text{ days} \approx \$5{,}800/\text{month}$$

*(Instance: **r6i.16xlarge** — 512 GB RAM, 64 vCPUs)*

You are not paying for compute. You are paying for RAM. The 64 CPU cores come along whether you need them or not.

Managed vector databases are not materially cheaper at this scale. For 100M float32 vectors on a production managed service, costs typically run $4,000 to $8,000 per month, depending on provider and availability requirements.

That is the baseline, before anything interesting has been built on top of it.

Layer 2 — The PCIe Bottleneck

GPU-accelerated vector search only becomes practical if the corpus fits in VRAM. At 100M float32 vectors, it does not. As shown above, moving 300 GB across PCIe 4.0 takes roughly 9 seconds per query. That ceiling makes GPU search impractical at float32 scale.

Layer 3 — Cache Starvation

Vector search on CPU is not compute-bound. It is memory-bound. Specifically, it is cache-starved:

$$L_1 \approx 0.5 \text{ ns}, \quad L_2 \approx 7 \text{ ns}, \quad \text{RAM} \approx 100 \text{ ns}$$

$$\text{RAM is} \approx 14 \times \text{ slower than } L_2, \quad \approx 200 \times \text{ slower than } L_1$$

[Reference: Latency Numbers Every Programmer Should Know — Jeff Dean]

At 3,072 bytes per vector, the L3 cache holds roughly 10,000 vectors; a rounding error on a 100M corpus. Every search traversal is a RAM access, not a cache access.

CPU cores are not waiting to compute dot products. They are waiting for vectors to arrive from RAM. You are paying for compute and using it as a waiting room.

Layer 4 — Throughput Collapse

All of the above compounds are under load. A single slow query is a latency problem. A thousand concurrent slow queries, each traversing a 300 GB index, each causing cache misses, each waiting on RAM, is a throughput collapse. p99 latency climbs. Infrastructure autoscales. The bill compounds.

This is the wall. Not a storage problem with storage symptoms; a storage problem with latency, throughput, bandwidth, and cost symptoms, all at once.

So What Do We Do?

Two techniques attack this problem from different angles: MRL and Quantization.

MRL (Matryoshka Representation Learning) puts semantic meaning into the first dimensions of an embedding. The model isn't spreading meaning evenly across all 768 dimensions. Instead, it trains the first 64 dimensions to capture the main semantic structure. Then, the later dimensions add more specific details. Think of nested dolls; each smaller doll is a complete picture, just coarser. On a standard model, slicing dimensions produces miscalibrated vectors. On an MRL-trained model, slicing is a design feature.

We are reducing the **number of dimensions**: the number of axes used to describe semantic meaning.

Quantization attacks the same problem differently. Instead of fewer dimensions, use fewer bits per dimension. Float32 gives each dimension 32 bits, which can represent:

$$2^{32} \approx 4.3 \text{ billion distinct values}$$

Binary quantisation replaces that with a single bit : 0 or 1, based on the sign of the value. We keep only the direction, positive or negative. In the embedding space, direction carries the semantic signal. Magnitude is mostly noise at retrieval time.

We are reducing the **precision per dimension**: how much space each axis gets.

The compression that results:

$$\text{768-dim float32} \rightarrow 3{,}072 \text{ bytes} \quad (1 \times \text{ baseline})$$

$$\text{64-dim float32} \rightarrow 64 \times 4 = 256 \text{ bytes} \quad (12 \times \text{ smaller})$$

$$\text{64-dim binary (packed)} \rightarrow \frac{64 \text{ bits}}{8} = 8 \text{ bytes} \quad (384 \times \text{ smaller})$$

One distinction worth making explicit: the 8-byte figure is the true 1-bit packed size, not the naive uint8 representation, which would be 64 bytes. Production systems (FAISS, Qdrant, Weaviate) use true 1-bit packing, and so does this experiment.

These two techniques work on independent axes. Combine them, and we go from 3,072 bytes to 8 bytes per vector. The question is what that costs in retrieval quality. That is what this experiment tests.

The Experiment

I analysed 20,000 actual Amazon Electronics product listings. This included titles, descriptions, and feature specs. I sampled them evenly across four categories: Computers and Laptops, Camera and Photo, Mobile and Phones, and Audio and Home Theater. Equal sampling is important. Without it, the retrieval metric favours memorising how often categories appear instead of grasping their meaning (you can read more about Class Imbalance). 5,000 products per category; no shortcuts.

The model: nomic-ai/nomic-embed-text-v1.5: specifically because it is natively MRL-trained. Truncating to 64 dimensions is a design intent, not a workaround. One key requirement: Apply F.layer_norm() before truncation in Nomic v1.5. Use a mandatory task prefix: search_document: for the corpus and search_query: for queries. Skipping either causes embeddings to silently degrade. It is also open weights with 8K context; the sensible choice for a reproducible local experiment.

What I compressed to:

Version	Dimensions	Bytes/ Vector	1M Vectors	Compression	Methodology
Baseline	768	3072	3.07 GB	1x	float32 (Full Precision)
MRL-256	256	1024	1.02 GB	3x	Truncate + LayerNorm
MRL-128	128	512	0.51 GB	6x	Truncate + LayerNorm
MRL-64	64	256	0.26 GB	12x	Truncate + LayerNorm
MRL+Binary	64	8	8.0 MB	384x	1-bit Sign + Bit-packing

How I measured quality: Recall@10. I picked 200 random products and found their 10 nearest neighbours in the float32 baseline. Then, I checked how many of those 10 neighbours each compressed version recovered.

$$\text{Recall@10} = \frac{|\text{neighbours}_{\text{compressed}} \cap \text{neighbours}_{\text{baseline}}|}{10}$$

A score of 1.0 means perfect agreement with the baseline. A score of 0.5 means the compressed version found 5 of the correct 10.

One caveat on this metric, which matters for interpreting the results: Recall@10 measures neighbour *identity* agreement, not semantic correctness. On a corpus of near-identical products, these are not the same thing.

📎 *Full pipeline: data loading, embedding generation, compression, evaluation, UMAP is on GitHub:* https://github.com/ria-19/Articles-code/blob/master/01-mrl-binary-compression/experiment.ipynb

The Results

Version	Bytes/vector	1M vectors (GB)	Compression	Recall@10	Quality retained
Baseline	3072	3.072	1×	1	100%
MRL-256	1024	1.024	3×	0.757	75.70%
MRL-128	512	0.512	6×	0.647	64.70%
MRL-64	256	0.256	12×	0.502	50.20%
MRL+Binary	8	0.008	384×	0.139	13.90%

The storage numbers are exact: actual numpy arrays, actual file sizes. The 384× compression is real.

The Drop Is Not Linear

$$3\times \text{ compression} \rightarrow \text{lost } 24.3$$

$$6\times \text{ compression} \rightarrow \text{lost a further } 11.0$$

$$12\times \text{ compression} \rightarrow \text{lost a further } 14.2$$

$$384\times \text{ compression} \rightarrow \text{lost a further } 36.3$$

The largest single drop is the final jump, from 64-dim float32 to 64-dim binary.

Going from 12× to 384× compression, a 32× additional storage reduction; costs 36 percentage points of recall. That is the binary quantisation tax.

Before deciding that binary is broken, it's important to know what Recall@10 measures in this dataset. This number can underestimate real quality in a specific and significant way.

What Recall@10 Is and Is Not

Recall@10 measures neighbour identity agreement, not semantic correctness. For each query, it asks: Did the compressed version find the same 10 products the baseline found?

On a corpus of near-identical products, these are not the same question.

Here is a concrete example:

Query: edaiser Touch Magic Keyboard Case for iPad Air 5th/4th Generation
Baseline top-5 retrieved:
1. Touchpad iPad 10.9 Air 5th Generation Case with Keyboard 2022
2. iPad Air 5th/4th Generation Case with Keyboard, Smart Trackpad
3. iPad Pro 11 inch 2021 Case with Keyboard for iPad Air 4
4. CHESONA iPad Pro 11 Case with Keyboard, Detachable
5. Qulose Keyboard Case for 10.9" iPad Air 5th 4th Generation
64-dim top-5 retrieved:
✓ iPad Air 5th/4th Generation Case with Keyboard, Smart Trackpad
✓ Touchpad iPad 10.9 Air 5th Generation Case with Keyboard 2022
✓ Keyboard Case for iPad Air 5th Generation 10.9" 2022
✓ iPad Pro 11 inch 2021 Case with Keyboard for iPad Air 4
✓ Qulose Keyboard Case for 10.9" iPad Air 5th 4th Generation

The 64-dim version retrieved 4 of the 5 correct neighbours. The miss, baseline rank 4 (CHESONA); was replaced by a product that is semantically identical. Both are iPad keyboard cases with trackpads. A user would be equally satisfied with either result. Recall counted it as a miss. It was not a miss.

The 50.2% recall for 64-dim float32 is therefore a lower bound on real quality, not a direct measure of it. The compressed version is not returning wrong products; it is returning equally valid products in a slightly different order. On a corpus of near-duplicate electronics accessories, Recall@10 is systematically pessimistic.

This does not apply to binary. Binary has genuine failures, not merely reranking.

Where Binary Actually Breaks

I examined the five worst binary retrievals; queries where binary found zero correct neighbours out of 10. Three distinct failure modes emerged:

Failure Mode 1 — Accessory confusion. Products defined by what they attach to; "case for iPad", "stand for MacBook"; lose the attachment context under binary compression. At 64 bits, an iPad case and an iPhone case are indistinguishable. The model knows it is an accessory. It has forgotten what it is an accessory for.

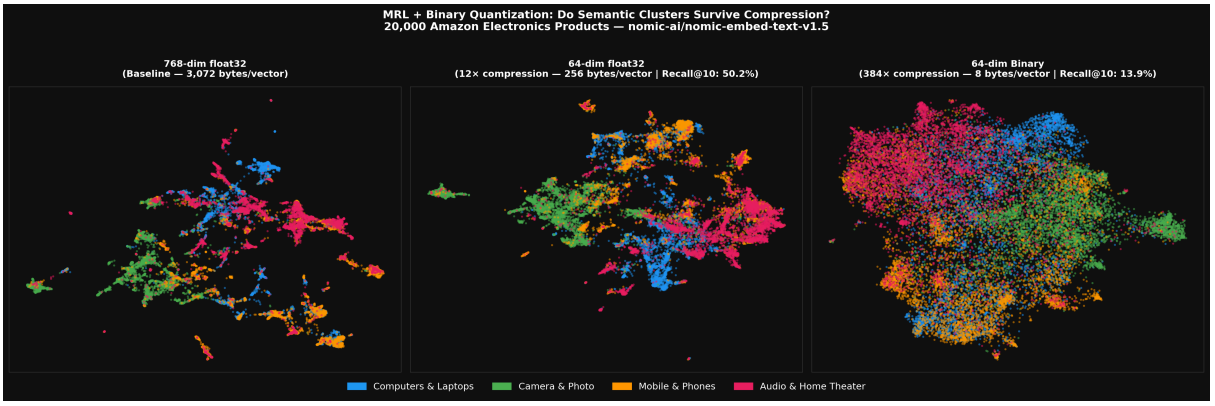
Failure Mode 2 — Polysemy collapse. The word "case" means a protective cover in one context and a computer enclosure in another. Float32 preserves this distinction through the surrounding context. Binary collapses it. A Silverstone ATX tower case retrieves camera cases and speaker enclosures because "case" was the dominant

signal that survived; the disambiguating context did not. The English language, it turns out, is not designed with 64-bit compression in mind.

Failure Mode 3 — Data contamination. One of the worst binary retrievals was a DIY drawing stencil and watercolour set, labelled as Audio and Home Theater in Amazon's original data. This is not a model failure. It is a data quality failure. Binary is being penalised for a miscategorisation that predates any compression.

Two of these three failures are genuine binary limitations. One is a data quality issue. The distinction matters when assessing whether binary is appropriate for a given use case.

The UMAP — Three Stories in One Image



Each dot is one product. Each colour is one category. UMAP takes 768-dimensional vectors and maps them into 2D. It keeps the neighbourhood structure, so similar products in high-dimensional space show up close in the plot.

Panel 1 (768-dim baseline): The clusters are separated but not perfectly isolated. Products bleed across category boundaries at the edges; an iPad keyboard case genuinely shares semantic space with phone accessories. Sub-clusters are visible within the main regions. This is honest geometry from a real product corpus, not the clean diagram you would draw on a whiteboard.

Panel 2 (64-dim float32): Something counterintuitive happens here. The clusters pulled *tighter* and more compact than the baseline. How does losing 704 dimensions make clusters cleaner?

This is the MRL front-loading effect. The first 64 dimensions carry the strongest category-level signal. The small differences that spread points along the baseline, like colour options for phone cases or slight spec changes in laptops, get lost. What

remains is the core semantic structure. The clusters appear more defined precisely because within-category noise has been removed. This is why 64-dim still finds the right category even when it does not find the exact product.

Panel 3 (64-dim binary): The cluster structure largely dissolves into a single mixed cloud. The four colours are still faintly organised if you look carefully; pink has regions, green has a corner, but the clean separation is gone. Products from different categories scatter across the same coordinates.

This is not random noise. It is a coarse category signal with no fine-grained structure. Binary can distinguish, roughly, that Audio products differ from Camera products. It has lost the ability to distinguish specific products within or near category boundaries. The UMAP makes visually explicit what the 13.9% recall figure implies numerically.

The Three Findings Together

Finding 1 — The compression table proves the storage claim. 384x is real. 3 GB becomes 8 MB per 1M vectors. Demonstrated on actual data with actual numpy arrays.

Finding 2 — The recall curve is honest but conservative. Quality drops faster than published benchmarks suggest on real product data. Two reasons: long texts (averaging 1,134 characters per product) hold more information in high dimensions. Also, a collection of similar products makes Recall@10 overly pessimistic. The numbers are accurate. They understate user-perceived quality for float32 truncations.

Finding 3 — Binary is a first-stage tool, not a final-stage tool. Binary preserves coarse category structure and loses fine-grained product distinction. It knows the department. It does not know the product. This makes it well-suited to first-stage retrieval in a two-stage pipeline; sweep billions of vectors in binary, rerank the top 100 in float32.

The Practical Answer — Use Both in Sequence

The answer is not "use binary" or "use float32." The answer is to use both in sequence:

Binary 64-dim sweeps the full corpus cheaply and retrieves the right neighbourhood. Float32 128-dim on the shortlist finds the right product within that neighbourhood.

At 100M vectors, the binary index sits at roughly:

100M × 8 bytes = 800 MB

That fits entirely in GPU VRAM. The float32 rerank runs on 100 candidates; negligible cost. Vector database expenditure drops by over 90%. Retrieval quality remains production-grade.

The two-stage pipeline is not a compromise. It is the architecture.

Limitations

What this experiment does not establish:

- Recall@10 measures neighbour agreement, not semantic correctness. Real quality on similar product corpora is likely higher than these figures suggest.
- Results are specific to one model (Nomic v1.5) and one domain (electronics). Different domains, like news headlines or legal clauses, may show different degradation curves.
- This ran on 20K samples on a free Colab T4. It is a quality experiment, not a latency benchmark. The actual speed difference between binary and float32 search at 10M vectors warrants a separate experiment.
- Binary search here used CPU XOR and popcount without SIMD hardware acceleration. Production Hamming distance with AVX-512 would be materially faster than these timings suggest.

Conclusion

We started with a storage problem: 3 GB to 600 GB of RAM just to hold vectors, before model weights, before indexes, before anything else. Then we asked: what if we compressed aggressively? Not a little; 384x aggressively.

The answer is that you can. The storage claim is real. The quality cost is real, but it focuses on certain failure modes. These include accessory-heavy, polysemous, and near-duplicate product data. You can tackle this with a low-cost two-stage pipeline.